arXiv:2310.18356v2 [cs.CL] 31 Oct 2023

LoRAShear: Efficient Large Language Model Structured Pruning and Knowledge Recovery

Tianyi Chen [1] **Tianyu Ding** [1] **Badal Yadav** [1] **Ilya Zharkov** [1] **Luming Liang** [1]
[1]Microsoft
{tiachen,tianyuding,bayadav,zharkov,lulian}@microsoft.com

Abstract

Large Language Models (LLMs) have transformed the landscape of artificial intelligence, while their enormous size presents significant challenges in terms of computational costs. We introduces LoRAShear, a novel efficient approach to structurally prune LLMs and recover knowledge. Given general LLMs, LoRAShear at first creates the dependency graphs over LoRA modules to discover minimally removal structures and analyze the knowledge distribution. It then proceeds progressive structured pruning on LoRA adaptors and enables inherent knowledge transfer to better preserve the information in the redundant structures. To recover the lost knowledge during pruning, LoRAShear meticulously studies and proposes a dynamic fine-tuning schemes with dynamic data adaptors to effectively narrow down the performance gap to the full models. Numerical results demonstrate that by only using one GPU within a couple of GPU days, LoRAShear effectively reduced footprint of LLMs by 20% with only 1.0% performance degradation and significantly outperforms state-of-the-arts. (Code will be public soon.)

1. Introduction

The advent of Large Language Models (LLMs) (Zhao et al., 2023; Hadi et al., 2023) has marked a significant milestone in evolution of artificial intelligence. These models, distinguished by their extensive parameter sizes, have demonstrated emergent abilities (Wei et al., 2022), catalyzing breakthroughs not only in the realm of natural language processing but also across tasks in various domains (Driess et al., 2023). This has opened up new possibilities for advancing towards Artificial General Intelligence (AGI) (Everitt et al., 2018; Bubeck et al., 2023). However, the enormous size of LLMs, typically ranging from tens to hundreds of billions of parameters (Touvron et al., 2023), incurs substantial computational costs of both processing power and memory requirements.

Structured pruning is an effective way to deliver compact DNNs via identifying and removing redundant structures then recovering the lost knowledge (Han et al., 2015; Chen et al., 2021b). However, its application onto LLMs is facing significant challenges, due to the requirements of massive computational resources and the unavailable training datasets of both pretraining and instructed fine-tuning datasets (Brown et al., 2020). Consequently, the paradigms could be largely categorized as pruning under *limited* or *full* resources. For the limited-resource setup, recent pruning works (Ma et al., 2023; Zhang et al., 2023; Sun et al., 2023) uses Low-Rank-Adaptor (LoRA) (Hu et al., 2021) during either pruning and instructed fine-tuning stage to reduce the resource requirements, yet still face significant performance degradation to the full LLMs. For the full-resouce setup, Sheared-LLaMA (Xia et al., 2023) conducts structured pruning on the original LLMs to directly achieve compact counterparts outperforming the equal sizes of LLMs trained from scratch, while requires significant GPU powers that might be not feasible for the public users.

We propose LoRAShear, a novel structured pruning framework for LLMs in the limited-resource setup to significantly bring down the performance gap between pruned LLMs to their full versions. Compared with the existing works, LoRAShear has two main advantages to better preserve and recover the lost knowledge during pruning. Firstly, we proposed a novel Lora Half-Space Projected Gradient (LHSPG) to enable progressive structured pruning with inherent knowledge transfer over LoRA modules. Secondly, we propose a dynamic knowledge recovery stage to perform multi-stage fine-tuning in manner of both pretraining and instructed fine-tuning. Additionally, LoRAShear is applicable onto general LLMs with conducting dependency graph analysis over LLMs with LoRA modules

[1]Microsoft, Redmond WA 98052, United States. Correspondence to: Tianyi Chen <tiachen@microsoft.com>, Luming Liang <lulian@microsoft.com>.

Preprint and ongoing work.

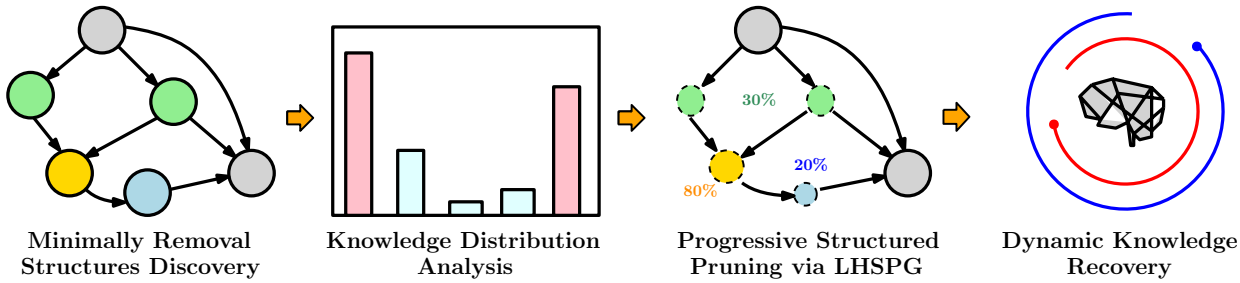


Figure 1. Overview of LoRAShear. Given a general LLM, LoRAShear at first discovers the minimally removal structures, then analyzes the knowledge distribution to mark the crucial ones as unprunable, then performs progressive structurally pruning over the prunable structures via LHSPG, and finally recovers the lost knowledge to recap the performance gap to the full LLM.

upon (Chen et al., 2023b).

We now summarize our main contributions as follows.

- **Dependency Graph Analysis on LLMs with LoRA Modules.** To automatically structurally prune general LLMs, discovering the minimally removal structures is necessary via dependency graph analysis. LLMs with LoRA poses additional challenges, since target structures are non-trainable yet auxiliary LoRA modules are learnable, which are ignored by existing algorithms (Chen et al., 2023b; Ma et al., 2023). We propose a novel graph algorithm to construct dependency graphs composed by overlapping node groups and composed node groups, and partition the trainable variables accordingly.
- **Progressive Structured Pruning via LHSPG.** We propose a novel structured sparsity optimization algorithm LoRA Half-Space Projected Gradient (LHSPG) to perform progressive structured pruning. LHSPG leverages the information from LoRA modules and effectively produces desired structured sparsity over the original variables. LHSPG transfers the knowledge stored in the relatively redundant structures to the important structures to better preserve the knowledge of the pretrained LLMs.
- **Dynamic Knowledge Recovery.** To further recover the knowledge after progressive pruning, we propose a dynamic knowledge recovery mechanism. Rather than only engaging the instructed fine-tuning as the existing limited-resource pruning works, we adaptively construct a subset from pretraining datasets upon the performance distribution to recover the lost general knowledge during pruning. We then perform the usual instructed fine-tuning to recover domain-specific expertise and the instruction capacity of pruned LLMs.
- **Experimental Results.** We demonstrate the effectiveness of LoRAShear on open-source LLAMAv1. By using one A100 GPU within a couple of GPU days, compared to the full model, the 20% pruned LLAMAv1 negligibly regresses 1% performance, and the 50% pruned LLAMAv1 preserves 82% performance on the evaluation benchmarks. Meanwhile, our results significantly outperform the existing state-of-the-arts.

2. Related Work

While pruning (Han et al., 2015) is well-established in traditional Deep Neural Networks (DNNs), its application to LLMs presents unique challenges. Unlike the smaller, task-specific DNNs (Ding et al., 2021; 2022), LLMs have a large number of parameters and require significant computational resources (Brown et al., 2020). Moreover, it's crucial for them to generalize well across multiple tasks (Xia et al., 2023). Recently, various pruning methods have been developed specifically for LLMs, generally falling into two main categories: unstructured and structured.

Unstructured Pruning. Unstructured pruning methods (Dong et al., 2017; Chen et al., 2020; 2021a) focus on setting unimportant individual weights in the model to zero. This fine-grained approach is straightforward and often maintains good performance, even with high compression rates. However, it results in sparse weight matrices that aren't well-suited for hardware accelerators, making them less efficient in real-world deployment. In the realm of LLMs, several new techniques have emerged. SparseGPT (Frantar & Alistarh, 2023) uses a sophisticated weight update process involving synchronized second-order Hessian updates, bypassing traditional retraining. In contrast, Wanda (Sun et al., 2023) achieves high sparsity without any retraining, simply by pruning weights with the smallest magnitudes multiplied by their corresponding input activations. PST (Li et al., 2022), however, combines unstructured pruning with efficient fine-tuning, pruning both LoRA and pre-trained model weights. A drawback of this method is the need for a memory-intensive mask that matches the shape of the pre-trained weights.

Structured Pruning. Structured pruning methods (Chen et al., 2021b; 2023a;b) focus on removing entire groups of parameters, such as neurons or layers, rather than in-

dividual weights. This group-level approach is hardware-friendly as it maintains dense weight matrices. The main challenge is selecting which structures to remove without compromising model performance. In the context of LLMs, several recent techniques aim for more efficient deployment and inference acceleration. For example, LLM-Pruner (Ma et al., 2023) proposes a dependency detection algorithm to identify and remove non-critical coupled structures, followed by a rapid post-training phase with limited data. However, this method is memory-intensive as it requires full gradient information and is not compatible with LoRA, necessitating a separate post-training phase for knowledge recovery. In contrast, LoRAPrune (Zhang et al., 2023) integrates LoRA with iterative structured pruning, achieving both parameter-efficient fine-tuning and direct hardware acceleration. This approach is also memory-efficient, relying only on LoRA's weights and gradients for pruning criteria, unlike LLM-Pruner, which uses full gradients. Most recently, Sheared-LLaMA (Xia et al., 2023) aims to prune the model to a target architecture defined by existing pre-trained models. It then trains the pruned model using dynamically loaded data, based on each domain's rate of loss reduction, leading to more efficient data usage and faster performance improvement. However, Sheared-LLaMA allocates considerable computational resources to subsequent pre-training for performance recovery.

In this work, we present LoRAShear, a method for efficient structured pruning of LLMs while recovers knowledge. Compared to the existing methods, our approach uniquely leverages a novel graph algorithm to create dependency graphs for both the original LLM *and* LoRA modules. We further introduce a structured sparsity optimization algorithm that utilizes information from LoRA modules to update weights, thereby enhancing knowledge preservation. Following pruning, we employ a dual-phase training approach involving both pre-training and fine-tuning to recover general and domain-specific knowledge effectively.

3. LoRAShear

LoRAShear dedicately designs a comprehensive end-to-end pipeline to compress pretrained LLMs and deliver efficient knowledge recovery. The outlined is stated as Algorithm 1. Given a general LLM $\mathcal{M}$, we at first analyze its architecture, create its dependency graph, and partition its trainable variables into a group set $\mathcal{G}$ following the discovered minimally removal structures (Section 3.1). We then analyze the knowledge distributed over the minimally removal structures to exclude the ones that highly impact the model performance from pruning (Section 3.2). Next, progressive structured pruning is performed over the prunable structures via our proposed LHSPG to identify redundant structures and transfer the knowledge stored in the redundant structures back onto the important counterparts (Section 3.3), and construct a compressed LLM $\mathcal{M}^*$ via automatically removing redundant structures (Section 3.4). The lost knowledge during pruning is then recovered via a dynamic knowledge recovery stage to recap the performance of the compressed $\mathcal{M}^*$ to the full LLM (Section 3.5).

Algorithm 1 Outline of LoRAShear.

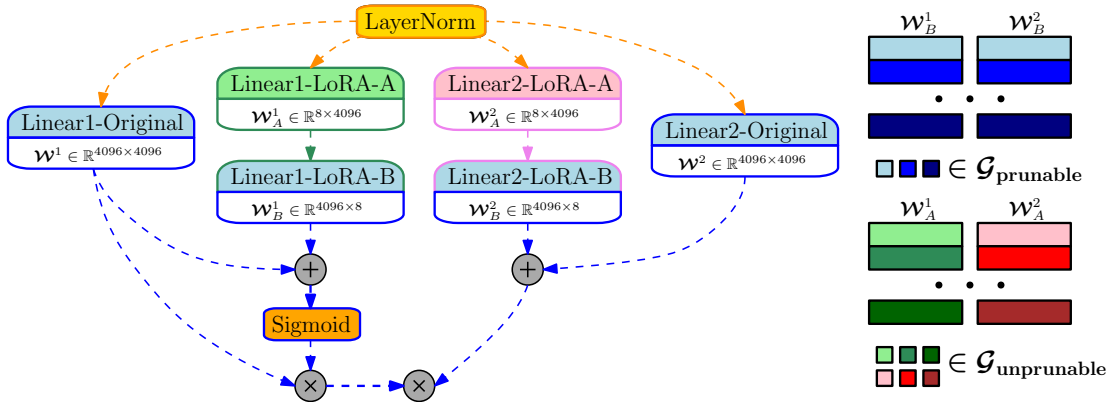
1: **Input.** A general pretraining LLM $\mathcal{M}$.
2: **Discover minimal removal structures** of $\mathcal{M}$ via creating and analyzing dependency graph $(\mathcal{V}, \mathcal{E})$. Partition trainable variables of $\mathcal{M}$ into $\mathcal{G}$.
3: **Analyze knowledge distribution** over each node group in the dependency graph.
4: **Progressive structured pruning by LHSPG** to identify redundant structures and transfer lost knowledge.
5: **Construct compressed model** to erasing redundancy to form compressed compact LLM $\mathcal{M}^*$.
6: **Dynamic fine-tuning** to recover lost knowledge.
7: **Output.** The compact high-performing LLM $\mathcal{M}^*$.

3.1. Minimally Removal Structure Discovery

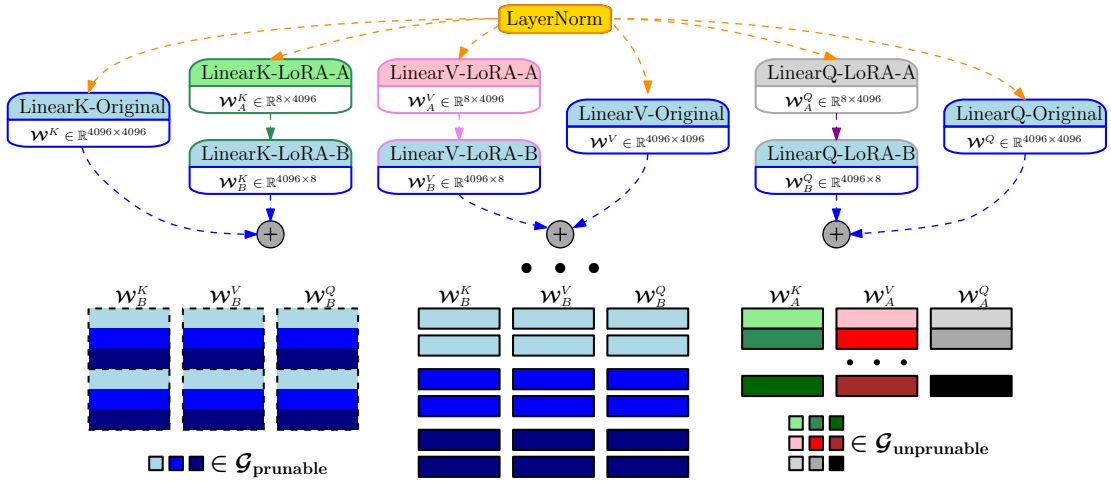
Algorithm 2 Minimally Removal Structure Discovery.

1: **Input.** A LLM $\mathcal{M}$ to be compressed and fine-tuned.
2: Construct the trace graph $(\mathcal{E}, \mathcal{V})$ of $\mathcal{M}$.
3: Establish node groups $\mathcal{N}_{\text{composed}}$ for composed operators via traversing $(\mathcal{E}, \mathcal{V})$ and the module tree of $\mathcal{M}$.
4: Establish node groups $\mathcal{N}_{\text{basic}}$ for remaining operators.
5: Build dependancy across $\mathcal{N}_{\text{composed}}$ and $\mathcal{N}_{\text{basic}}$.
6: Partition trainable variables into minimally removal structures and form $\mathcal{G}$.
7: **Return** the trainable variable groups $\mathcal{G}$.

Given a target LLM $\mathcal{M}$, the foremost step is to discover the minimally removal structures, which are defined as the units that can be directly removed without affecting the functionality of the remaining DNNs. Such discovery was achieved by analyzing the trace graphs and creating the dependency graphs over the basic operators in OTOv2 (Chen et al., 2023b). In these dependency graphs, each node group indicates the operators that are dependent and needs to be pruned together if with trainable variables and are disjoint to each other in the normal DNNs. However, LLMs with LoRA modules easily disrupt those algorithms since in such models, only LoRA modules are trainable, and the original LLM variables are fixed yet prunable. To address issue, we dedicately introduce composed operator and overlapping node groups. Composed operator refers to the operators that are assembled by multiple basic operators such as LoRA modules consisting of two `linear` operators, yet needs to be considered as an entirety. Each



(a) Dependency graph for MLP layers.



(b) Dependency graph for Attention layers.

Figure 2. Dependency graph in LLAMAv1 and trainable variable partitions.

such composed operator should form one node group, such as `Linear-1-LoRA-A-Linear-1-LoRA-B` in Figure 2a. The overlapping node groups $\mathcal{N}_{\text{composed}}$ exist because their outgoing nodes still need to obey the dependency across other adjacent operators in the trace graph, *e.g.*, `Linear-1-LoRA-B` belonging to two node groups marked as green and blue simultaneously. We then jointly consider basic and composed node groups $\mathcal{N}_{\text{basic}}$ and $\mathcal{N}_{\text{composed}}$ and partition the trainable variables of $\mathcal{M}$ into a set of groups $\mathcal{G}$, wherein each group $g \in \mathcal{G}$ corresponds to one minimally removal structure.

3.2. Knowledge Distribution Analysis

Due to the universal training process, the knowledge is unevenly distributed across all the node groups in the dependency graph. Some node groups serve remarkably more significant roles than others, resulting in performance collapse if pruning them. Meanwhile, in the limited resources setting, the knowledge after collapse would not be easily recovered. Therefore, before engaging into the progressive structured pruning stage, we analyze the knowledge distribution to locate the node groups that should be excluded from pruning. As stated in Algorithm 3, we iter-

Algorithm 3 Knowledge Distribution Analysis.

Input. Trainable variable partition $\mathcal{G}$, node groups $\mathcal{N}_{\text{composed}} \cup \mathcal{N}_{\text{basic}}$, a set of pruning ratios $\mathcal{P}$, an evaluation dataset $\mathcal{D}_{\text{eval}}$, and a unprunable ratio γ.
for each node group in $\mathcal{N}_{\text{composed}} \cup \mathcal{N}_{\text{basic}}$ **do**
  Prune groups upon some specified proxy and $\mathcal{P}$.
  Compute performance deviation upon $\mathcal{D}_{\text{eval}}$.
  Recover pruned groups to the original status.
end for
Sort the perform deviation over $\mathcal{N}_{\text{composed}} \cup \mathcal{N}_{\text{basic}}$.
Mark the groups in $\mathcal{G}$ regarding the node groups with the largest deviation upon γ as unprunable $\mathcal{G}_{\text{unprunable}}$.
Mark the remaining groups in $\mathcal{G}$ as prunable $\mathcal{G}_{\text{prunable}}$.
Return prunable and unprunable variable groups $\mathcal{G}_{\text{prunable}} \cup \mathcal{G}_{\text{unprunable}}$.

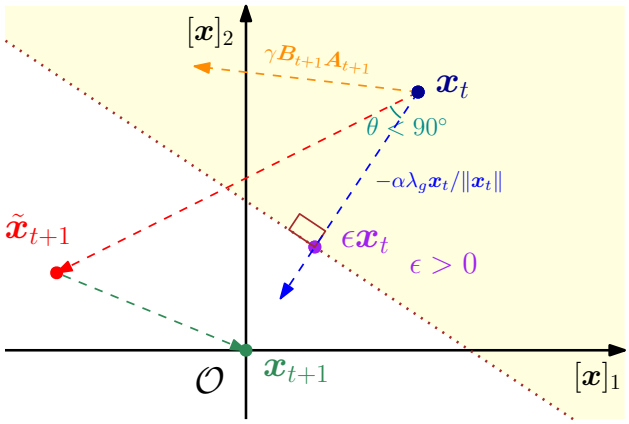


Figure 3. Half-Space step over LoRA modules.

atively traverse all node groups, and prune each of them upon some specified pruning ratio yet keep the remaining groups unchanged. We then evaluate the output deviation between each pruned LLM against the full model upon some pre-selected evaluation dataset. The ones with the largest $\gamma|\mathcal{N}_{\text{composed}} \cup \mathcal{N}_{\text{basic}}|$ deviations are marked as unprunable, which corresponding groups of variables to form $\mathcal{G}_{\text{unprunable}}$. The remaining ones are marked as prunable, where trainable variables form $\mathcal{G}_{\text{prunable}}$.

3.3. Progressive Structured Pruning via LHSPG

The next step is to proceed progressive structured pruning over the prunable groups of variables $\mathcal{G}_{\text{prunable}}$. To proceed it, we propose a novel structured sparsity optimizer LoRA Half-Space Projected Gradient (LHSPG) to yield structured sparsity over the original model parameters based on the optimization information over auxiliary LoRA modules. There exist two main takeaways of LHSPG, *i.e.*, *(i)* effectively identify and remove redundant structures via projecting them onto zero, and *(ii)* transfer the knowledge stored in the relatively redundant structures to be pruned back to the important counterparts to better preserve the knowledge of full LLMs.

Target Problem. We formulate the progressive structured pruning as the following structured sparsity optimization problem (3) over LLMs with LoRA modules.

$$\underset{\mathcal{A},\mathcal{B}}{\text{minimize}}\ f(\mathcal{A},\mathcal{B}),\ \text{ s.t. Card}\{g \in \mathcal{G}_{\text{prunable}} | [\boldsymbol{x}]_g = 0\} = K, \tag{3}$$

where $\mathcal{A}$ and $\mathcal{B}$ are the collections of LoRA decomposing sub-matrices, which are trainable during the structured pruning. We seek to yield group sparsity over the original variables with the target sparsity level as K.

Outline. The outline of LHSPG is presented in Algorithm 4. We at first warm up the LoRA variables in the prunable groups $\mathcal{G}_{\text{prunable}}$ via stochastic gradient descent (SGD) or its variants like AdamW to collect gradient information. We then progressively identify redundant structures within P periods of sparse optimization. To proceed, we compute the target group sparsity level to be produced for each period. In each period p, we sort the prunable groups upon some prespecified saliency proxies and pick up the ones with least saliency scores as redundant groups for the current period $\widehat{\mathcal{G}}_p$. Then we compute trial iterate over the LoRA variables in $\mathcal{A}$ and $\mathcal{B}$ via SGD or its variants. For the redundant groups $\widehat{\mathcal{G}}_p$, we proceed a gradient descent via LoRA approximation and penalize over the variable magnitude proportionally to λ_g, which is selected upon the length of each pruning period. A Half-Space projection is next performed over the trial iterate to project groups of variables with the least sacrificing over the objective function. During the whole process, redundant groups are progressively projecting onto zero, during the projec-

Algorithm 4 Progressive Structured Pruning via LHSPG

Input. pretraining variable $\boldsymbol{x}_0$, learning rate α, warm-up steps T_w, progressive period P, period length T_p, target group sparsity level K, and variable partition $\mathcal{G}_{\text{prunable}} \cup \mathcal{G}_{\text{unprunable}}$.
Warm up T_w steps via SGD or its variants (AdamW).
Initialize redundant groups $\mathcal{G}_{\text{redundant}} \leftarrow \emptyset$.
Initialize important groups $\mathcal{G}_{\text{important}} \leftarrow \mathcal{G}$.
Compute sparsity level for each pruning period $\widehat{K} := K/T_p$.
for each pruning period $p = 0, 1, \cdots, P-1$ **do**
  Pickup $\widehat{\mathcal{G}}_p$ in $\mathcal{G}_{\text{important}}$ with $\widehat{K}$-least saliency scores.
  Update $\mathcal{G}_{\text{redundant}} \leftarrow \mathcal{G}_{\text{redundant}} \cup \widehat{\mathcal{G}}_p$.
  Update $\mathcal{G}_{\text{important}} \leftarrow \mathcal{G}_{\text{important}} / \widehat{\mathcal{G}}_p$.
  for $t = 0, 1, \cdots, T_p - 1$ **do**
    Update LoRA $\boldsymbol{B}$ and $\boldsymbol{A}$ via SGD or its variants.

$$\begin{aligned} \boldsymbol{B}_{t+1} &\leftarrow \boldsymbol{B}_t - \alpha_k \nabla_{\boldsymbol{B}_t} f \\ \boldsymbol{A}_{t+1} &\leftarrow \boldsymbol{A}_t - \alpha_k \nabla_{\boldsymbol{A}_t} f \end{aligned} \tag{1}$$

    Compute trial iterate $[\tilde{\boldsymbol{x}}_{t+1}]_{\widehat{\mathcal{G}}_p}$ for each $g \in \widehat{\mathcal{G}}_p$.

$$[\tilde{\boldsymbol{x}}_{t+1}]_g \leftarrow [\boldsymbol{x}_t + \gamma \boldsymbol{B}_{t+1}\boldsymbol{A}_{t+1}]_g - \frac{\lambda_g [\boldsymbol{x}_t]_g}{\|[\boldsymbol{x}_t]_g\|} \tag{2}$$

    Perform Half-Space projection over $[\tilde{\boldsymbol{x}}_{t+1}]_{\widehat{\mathcal{G}}_p}$.
    Update $[\boldsymbol{x}_{t+1}]_{\widehat{\mathcal{G}}_p} \leftarrow [\tilde{\boldsymbol{x}}_{t+1}]_{\widehat{\mathcal{G}}_p}$.
    Update $[\boldsymbol{B}_{t+1}]_{\widehat{\mathcal{G}}_p} \leftarrow \boldsymbol{0}$.
    if $t = T_p - 1$ **then**
      Merge $[\boldsymbol{B}_{t+1}\boldsymbol{A}_{t+1}]_{\mathcal{G}_{\text{important}}}$ into $[\boldsymbol{x}]_{\mathcal{G}_{\text{important}}}$.
    end if
  end for
end for
Return the final iterate $\boldsymbol{x}^*_{\text{LHSPG}}$.

tion, the LoRA modules for the important counterparts are absorbing the knowledge via minimizing the loss functions. As a result, the progressive structured pruning not only effectively identifies and projects redundant groups of variables onto zero, but also preserve the knowledge stored in the redundant structures to the largest extent. A final iterate $\boldsymbol{x}^*_{\text{LHSPG}}$ is returned for the subsequent step.

3.4. Compressed LLM Construction

Given the solution of LHSPG, LoRAShear automatically constructs a structurally pruned LLM $\mathcal{M}^*$ via automatically erasing the structures corresponding to the redundant groups in $\mathcal{G}_{\text{prunable}}$. The whole procedure is performed via two pass dependency graph traversal. The first-pass traversal iterates each node group and prunes the structures along the primary dimension. The second-pass traversal erases the structures along the secondary dimension upon the pruned status of the incoming structures.

3.5. Dynamic Knowledge Recovery

Algorithm 5 Dynamic Knowledge Recovery.

Input. pretraining dataset $\mathcal{D}_{\text{pretraining}}$, instructed fine-tuning dataset $\mathcal{D}_{\text{instruct}}$, and a pruned LLM $\mathcal{M}^*$.
Establish validation datasets for $\mathcal{D}_{\text{pretraining}}$ and $\mathcal{D}_{\text{instruct}}$ as $\mathcal{D}^{\text{val}}_{\text{pretraining}}$ and $\mathcal{D}^{\text{val}}_{\text{instruct}}$, respectively.
for $\mathcal{D} \in \{\mathcal{D}_{\text{pretraining}}, \mathcal{D}_{\text{instruct}}\}$ **do**
  while not converge **do**
    Dynamically construct $\widehat{\mathcal{D}} \subseteq \mathcal{D}$ by evaluation.
    Fine-tune $\mathcal{M}^*$ with LoRA on $\widehat{\mathcal{D}}$.
  end while
end for
Return knowledge-recovered pruned LLM $\mathcal{M}^*$.

The final step is recovering lost knowledge after pruning and restoring the capabilities of LLM. To achieve successful recovery, it's essential to understand how the LLM acquires its knowledge. The knowledge is acquired through a two-stage process: pretraining on extensive and diverse text corpora, followed by fine-tuning with specific instruction datasets. The acquired knowledge is stored as variables within the LLM, but these variables are removed during the pruning process. Therefore, to regain the knowledge, a post-training process is required, involving both the pretraining and instructed fine-tuning datasets.

Due to the vast and diverse nature of the pretraining datasets, existing structured pruning methods, especially those designed for limited resources, only rely on the instructed fine-tuning datasets. However, this approach often leads to a significant degradation in performance. To mitigate this challenge, we introduce a dynamic knowledge recovery framework, presented as Algorithm 5.

In particular, given the pretraining and the instructured fine-tuning dataset collections $\mathcal{D}_{\text{pretraining}}$ and $\mathcal{D}_{\text{instruct}}$, we at first uniformly sample subsets from them for validation $\mathcal{D}^{\text{val}}_{\text{pretraining}}$ and $\mathcal{D}^{\text{val}}_{\text{instruct}}$. We then consider knowledge recovery over pretraining datasets. To proceed, we at first evaluate the performance degradation over the different sources via $\mathcal{D}^{\text{val}}_{\text{pretraining}}$. Upon on the performance deviation distribution, we construct a subset $\widehat{\mathcal{D}}_{\text{pretraining}} \subseteq \mathcal{D}_{\text{pretraining}}$. The criteria for selecting samples involve prioritizing categories experiencing more significant degradation while ensuring a balanced representation of samples from sources with minimal degradation to prevent overfitting. Subsequently, we employ LoRA for fine-tuning the pruned model. If the evaluation results do not converge, we repeat the process of constructing the next subset from $\mathcal{D}_{\text{pretraining}}$ until convergence is achieved. Following the knowledge recovery from the pretraining stage, we apply the same methodology to the instructed fine-tuning datasets. This iterative approach ultimately yields the highly optimized pruned LLM $\mathcal{M}^*$.

4. Numerical Experiments

To demonstrate the effectiveness of LoRAShear, we provide our preliminary results on open-source LLAMAv1 (Touvron et al., 2023). More experimental results will come in the future revisions.

4.1. Dataset Selection

Pretraining Datasets. We follow Touvron et al. to collect pretraining datasets or the alternatives for English. In particular, we select the OpenWebText (Aaron Gokaslan, 2019) as an alternative to English CommonCrawl and C4 datasets. We select a processed Wikipedia dump on 2022 (Foundation). Gutenberg (Gerlach & Font-Clos, 2020) and BookCorpus (Zhu et al., 2015) are also used in our collection. For each datasets, we proceed standard pre-processing to erase irregular characters and only keep the paragraphs that contains more than 64 tokens.

Instructed Fine-Tuning Datasets. For fair comparison, we follow the existing structured pruning LLM works (Ma et al., 2023; Zhang et al., 2023) in the limited-resource setting to use the Alpaca dataset (Taori et al., 2023), which consists 52,000 instructions and demonstrations generated by OpenAI's text-davinci-003 engine. Alpaca is frequently used to conduct general instruction-tuning for language models and make the LLM follow instruction better.

4.2. Experimental Results

Knowledge Distribution Analysis. The analyzed knowledge distribution on LLAMAv1 is presented in Figure 4. Given an evaluation dataset, we perform Algorithm 3 to analyze the knowledge distributed across the minimally re-

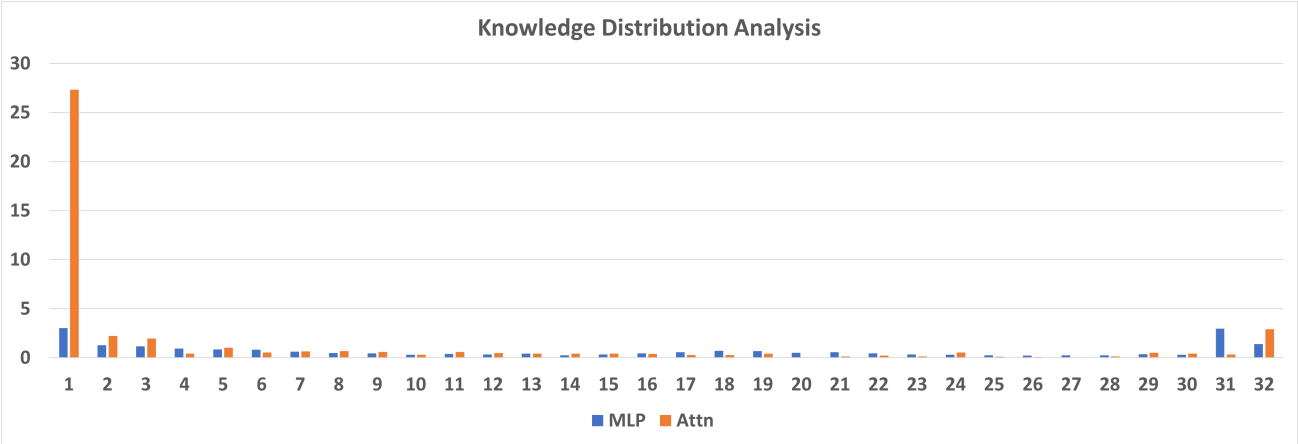


Figure 4. Knowledge distribution analysis by measuring the perplexity deviation to the full LLAMAv1.

Table 1. LoRAShear over LLAMAv1.

Pruning Ratio	Method	BoolQ	PIQA	HellaSwag	WinoGrande	ARC-e	ARC-c	OBQA	Average
Ratio = 0%	LLAMAv1 (Touvron et al., 2023)	76.5	79.8	76.1	70.1	72.8	47.6	57.2	68.59
(Baseline)	LLAMAv1 (Ma et al., 2023)	73.18	78.35	72.99	67.01	67.45	41.38	42.40	63.25
Ratio = 20%	LLM-Pruner (Ma et al., 2023)	66.79	77.58	68.48	64.96	64.06	37.88	39.00	59.82
	LoRAPrune (Zhang et al., 2023)	65.82	79.31	70.00	62.76	65.87	37.69	39.14	60.05
	WANDA (Sun et al., 2023)	65.75	74.70	64.52	59.35	60.65	36.26	39.40	57.23
	LoRAShear†	70.17	76.89	68.69	65.83	64.11	38.77	39.97	60.63
	LoRAShear	72.78	76.36	69.49	67.63	69.02	39.47	40.78	**62.22**
Ratio = 50%	LLM-Pruner (Ma et al., 2023)	61.56	68.72	46.62	52.64	47.94	29.27	35.40	48.88
	LoRAPrune (Zhang et al., 2023)	61.88	71.53	47.86	55.01	45.13	31.62	34.98	49.71
	WANDA (Sun et al., 2023)	50.90	57.38	38.12	55.98	42.68	34.20	38.78	45.43
	LoRAShear†	62.12	71.80	48.01	56.29	47.68	32.26	34.61	50.39
	LoRAShear	63.40	72.15	49.83	56.40	49.45	34.31	35.86	51.63

† Knowledge recovery only on the instructured fine-tuning datasets as other works.

moval structures in each node group. After measuring the output deviation, it is apparent that the knowledge is unevenly distributed across different node groups. The first and last few node groups serve as more significant roles than others to the model prediction. During pruning, it would be better to avoid pruning these most sensitive node groups since the saliency score calculation may still prune some of their minimally removal structures which may result in significant performance degradation.

Pruning Results. We now show the quantitative results of LoRAShear and compare with other methods over the evaluation benchmark computed via `lm-evaluation-harness` (Gao et al., 2021). As shown in Table 1, under the same pruning ratio 20%, LoRAShear significantly outperforms others by 2.2%-5.0% accuracy and negligibly regress 1% compared to the full LLAMAv1. We additionally conduct an ablation that only levering the same instructured fine-tuning dataset, *i.e.*, Alpaca, to recover the lost knowledge. The performance in this setting still outperform other methods, implying the effectiveness of progressive structured pruning via LHSPG to transferring and preserving knowledge. Under the high pruning ratio 50%, the outerperformance of LoRAShear still holds. In particular, under both progressive structured pruning followed by knowledge recovery via pretraining and instructured fine-tuning datasets, our performance is significantly better than the existing state-of-the-arts.

5. Conclusion

We propose a novel LoRAShear to conduct efficient structured pruning and knowledge recovery for general LLMs in the limited resources setup. LoRAShear has three takeaways: *(i)* it automatically discovers minimally removal structures over LLMs with LoRA modules; *(ii)* conducts progressive structured pruning via a novel structured sparsity optimizer LHSPG that yields structured sparsity over original variables via the information stored in LoRA modules; and *(iii)* equips with a dynamic knowledge recovery stage to gain knowledge back from both pretraining and instructured fine-tuning datasets. Numerical results validates the efficacy, that negligibly regress 1% performance to the full model under 20% pruning ratio and preserve 82% performance under 50% pruning ratio to the full LLMs. More experiments will come in the updated versions.

References

Aaron Gokaslan, Vanya Cohen, E. P. S. T. Openwebtext corpus, 2019.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., Lee, P., Lee, Y. T., Li, Y., Lundberg, S., et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.

Chen, T., Ji, B., Shi, Y., Ding, T., Fang, B., Yi, S., and Tu, X. Neural network compression via sparse optimization. *arXiv preprint arXiv:2011.04868*, 2020.

Chen, T., Ding, T., Ji, B., Wang, G., Shi, Y., Tian, J., Yi, S., Tu, X., and Zhu, Z. Orthant based proximal stochastic gradient method for l1-regularized optimization. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2020, Ghent, Belgium, September 14–18, 2020, Proceedings, Part III*, pp. 57–73. Springer, 2021a.

Chen, T., Ji, B., Ding, T., Fang, B., Wang, G., Zhu, Z., Liang, L., Shi, Y., Yi, S., and Tu, X. Only train once: A one-shot neural network training and pruning framework. In *Advances in Neural Information Processing Systems*, 2021b.

Chen, T., Liang, L., Ding, T., and Zharkov, I. Towards automatic neural architecture search within general super-networks. *arXiv preprint arXiv:2305.18030*, 2023a.

Chen, T., Liang, L., Ding, T., Zhu, Z., and Zharkov, I. Otov2: Automatic, generic, user-friendly. *arXiv preprint arXiv:2303.06862*, 2023b.

Ding, T., Liang, L., Zhu, Z., and Zharkov, I. Cdfi: Compression-driven network design for frame interpolation. *arXiv preprint arXiv:2103.10559*, 2021.

Ding, T., Liang, L., Zhu, Z., Chen, T., and Zharkov, I. Sparsity-guided network design for frame interpolation. *arXiv preprint arXiv:2209.04551*, 2022.

Dong, X., Chen, S., and Pan, S. Learning to prune deep neural networks via layer-wise optimal brain surgeon. *Advances in neural information processing systems*, 30, 2017.

Driess, D., Xia, F., Sajjadi, M. S., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.

Everitt, T., Lea, G., and Hutter, M. Agi safety literature review. *arXiv preprint arXiv:1805.01109*, 2018.

Foundation, W. Wikimedia downloads. URL https://dumps.wikimedia.org.

Frantar, E. and Alistarh, D. SparseGPT: Massive language models can be accurately pruned in one-shot. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 10323–10337. PMLR, 23–29 Jul 2023.

Gao, L., Tow, J., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., McDonell, K., Muennighoff, N., Phang, J., Reynolds, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. A framework for few-shot language model evaluation, September 2021. URL https://doi.org/10.5281/zenodo.5371628.

Gerlach, M. and Font-Clos, F. A standardized project gutenberg corpus for statistical analysis of natural language and quantitative linguistics. *Entropy*, 22(1):126, 2020.

Hadi, M. U., Qureshi, R., Shah, A., Irfan, M., Zafar, A., Shaikh, M. B., Akhtar, N., Wu, J., Mirjalili, S., et al. A survey on large language models: Applications, challenges, limitations, and practical usage. 2023.

Han, S., Mao, H., and Dally, W. J. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Li, Y., Luo, F., Tan, C., Wang, M., Huang, S., Li, S., and Bai, J. Parameter-efficient sparsity for large language models fine-tuning. In Raedt, L. D. (ed.), *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*, pp. 4223–4229. International Joint Conferences on Artificial Intelligence Organization, 7 2022.

Ma, X., Fang, G., and Wang, X. Llm-pruner: On the structural pruning of large language models. *arXiv preprint arXiv:2305.11627*, 2023.

Sun, M., Liu, Z., Bair, A., and Kolter, J. Z. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*, 2023.

Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., and Hashimoto, T. B. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.

Xia, M., Gao, T., Zeng, Z., and Chen, D. Sheared llama: Accelerating language model pre-training via structured pruning. *arXiv preprint arXiv:2310.06694*, 2023.

Zhang, M., Shen, C., Yang, Z., Ou, L., Yu, X., Zhuang, B., et al. Pruning meets low-rank parameter-efficient fine-tuning. *arXiv preprint arXiv:2305.18403*, 2023.

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

Zhu, Y., Kiros, R., Zemel, R., Salakhutdinov, R., Urtasun, R., Torralba, A., and Fidler, S. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *The IEEE International Conference on Computer Vision (ICCV)*, December 2015.